Jerry

Enterprise Knowledge Assistant

Software Architecture and User Documentation

A technical reference for the local demonstration application

Document revision:	1.1
Reference date:	November 2026
Application branch model:	feature development through testing and production
Primary interfaces:	Streamlit user interface and FastAPI service

This document describes the repository implementation and its demo limitations.

Document control

Item	Description
Purpose	Explain the architecture, installation, operation, interfaces, safeguards, and test evidence for Jerry.
Audience	Project maintainers, evaluators, developers, and users running the local demo.
System scope	Python packages at the repository root, the Streamlit UI, FastAPI API, local sample documents, and optional hosted adapters.
Status	Assignment/demo reference. This is not a production security or service-level commitment.
Source set	The README, Markdown design notes in `docs/`, configuration template, API schemas, and the bundled example corpus.

How to use this document

The table of contents is linked. Select a chapter title in the PDF to navigate to that section. The document is organized from system context and setup through runtime behavior, interfaces, security, testing, and operational limitations. Code font marks file names, commands, environment variables, endpoints, and schema fields.

Contents

Document control i

How to use this document ii

1 Executive overview 1

1.1 Goals . . . 1

1.2 Important scope limits . . . 1

2 System architecture 2

2.1 Component model . . . 2

2.2 Development container topology . . . 2

2.3 Package responsibilities . . . 2

2.4 Branch workflow . . . 3

3 Installation and operation 5

3.1 Prerequisites . . . 5

3.2 Install dependencies and configure . . . 5

3.3 Build the local sample index . . . 5

3.4 Start the API and interface . . . 6

3.5 Run the development stack with Docker Compose . . . 6

3.6 Use the chat interface . . . 6

3.7 Refresh or select an index backend . . . 7

4 Configuration 8

5 Document ingestion and data model 10

5.1 Source records . . . 10

5.2 Chunking and indexing . . . 10

6 Search and retrieval 11

6.1 Search modes . . . 11

6.2 Metadata filters and citations . . . 11

7 Assistant workflow and analysis **12**
7.1 Grounded chat graph . . . 12
7.2 Recursive-analysis path . . . 12

8 Session memory, roles, and tools **13**
8.1 Session memory . . . 13
8.2 Role-based access control . . . 13
8.3 Tools . . . 13

9 API reference **14**
9.1 Chat request example . . . 14

10 Security and guardrails **16**
10.1 Trust boundaries . . . 16
10.2 Limitations and safe use . . . 16

11 Reliability and observability **17**
11.1 Rate limits . . . 17
11.2 Timeouts and error mapping . . . 17
11.3 LangSmith and streamed activity . . . 17

12 Testing and verification **19**
12.1 Recorded local result . . . 19
12.2 Coverage not established . . . 20
12.3 Docker development verification . . . 20
12.4 Repeat the manual checks . . . 20

13 Troubleshooting **21**

14 Deployment considerations **22**

15 Glossary and source documents **23**
15.1 Glossary . . . 23
15.2 Repository documentation references . . . 23

A Compile this LaTeX source **25**

Chapter 1

Executive overview

Jerry is a demonstration enterprise knowledge assistant for questions about internal policies, incident records, runbooks, and architecture notes. The user interface is built with Streamlit. A FastAPI service authenticates demo requests, applies rate limits and request validation, and calls a LangGraph workflow. The assistant retrieves passages from a small indexed corpus, applies evidence and citation checks, and returns an answer with source identifiers and workflow activity.

The default configuration is local-first. Sample Markdown documents are chunked and indexed into a local JSON Lines file. The default local feature-hash embedding is a repeatable demo representation; it is not a semantic embedding. With the local embedding provider, search uses BM25 keyword retrieval. OpenAI embeddings, Pinecone, an OpenAI chat model, and LangSmith tracing are optional integrations.

1.1 Goals

The implementation demonstrates:

- document loading, metadata validation, chunking, indexing, and source-aware retrieval;
- explicit assistant routing, evidence checking, bounded retry, grounded responses, and citation validation;
- bounded incident analysis, session memory, server-side demo-role checks, and local tool wrappers;
- input validation, prompt-injection screening, per-user rate limiting, safe error responses, trace IDs, and streamed activity.

1.2 Important scope limits

Jerry is not a production banking assistant. Demo accounts are token-mapped sample identities, memory and rate limits are process-local, the local corpus is fictional and small, the local RLM analysis uses deterministic text patterns, and MCP-style data is served by local wrappers rather than a connected MCP server. Guardrails reduce risk but do not prevent all prompt injection or misuse. External services must be separately configured and evaluated.

Chapter 2

System architecture

2.1 Component model

The Python package boundary is the repository root; the packages are not nested under an `app/` directory. The principal components and runtime paths are shown in Figure 2.1.

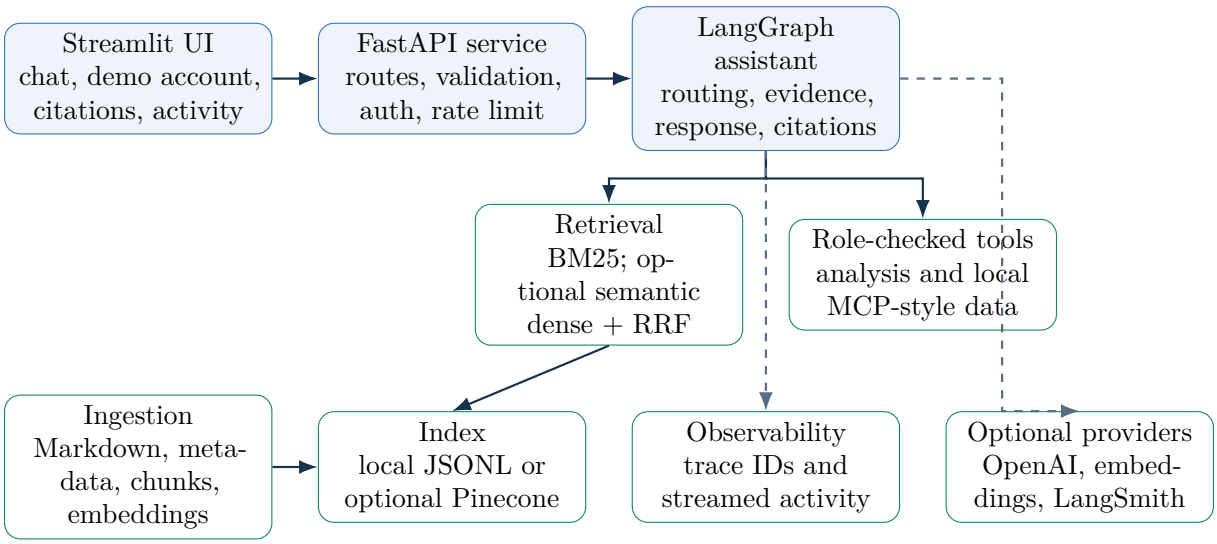


Figure 2.1: Jerry components and the primary request, retrieval, and data paths. Solid arrows show the main runtime flow; dashed arrows mark supporting or optional integrations.

2.2 Development container topology

Docker Compose defines `api`, `ui`, and `ingest` services from the root `Dockerfile`. API and UI bind-mount project source at `/workspace`; Uvicorn uses `-reload`. Compose publishes the API on host port 8010 and Streamlit on 8501, configurable with `JERRY_API_PORT` and `JERRY_UI_PORT`. The UI calls the API at `http://api:8010`; the API health check gates UI startup. Ingestion uses local embeddings and indexing, writing `data/local_index.jsonl` to the bind mount. This development setup is not production-hardened.

2.3 Package responsibilities

Package	Responsibility	Representative modules
`api/`	HTTP application, routes, dependencies, exception mapping	`api/main.py`; `api/routes/chat.py`; `api/routes/tools.py`
`core/`	Settings, logging, error types, token bucket	`core/config.py`; `core/rate_limit.py`
`models/`	Validated API, document, tool, and graph-state schemas	`models/api.py`; `models/documents.py`; `models/state.py`
`agent/`	LangGraph construction, routing, prompts, memory, analysis path	`agent/graph.py`; `agent/nodes.py`; `agent/rlm.py`
`retrieval/`	Ingestion, chunking, embeddings, dense/sparse/hybrid search	`retrieval/ingest.py`; `retrieval/sparse.py`; `retrieval/hybrid.py`
`tools/`	Search, Python analysis, permission gates, local MCP-style wrappers	`tools/knowledge_search.py`; `tools/analysis.py`; `tools/mcp_client.py`
`security/`	Demo authentication, request/content checks, response guardrails	`security/auth.py`; `security/validation.py`; `security/guardrails.py`
`observability/`	LangSmith setup and activity event schemas	`observability/tracing.py`; `observability/events.py`
`ui/`	Streamlit user interface and API streaming client	`ui/streamlit_app.py`; `ui/api_client.py`
`data/`	Fictional Markdown sample corpus and generated local index	`data/sample_docs/`; `data/local_index.jsonl`
`scripts/`	Developer utilities, including sample corpus ingestion	`scripts/ingest_sample_data.py`

2.4 Branch workflow

Feature implementation follows the integration path shown in Figure 2.2. The repository documentation identifies `main` as the default branch and the old `Proction` spelling as a legacy branch name. Branch pointers can differ between local and remote repositories; this diagram describes the intended process, not the current Git synchronization state.

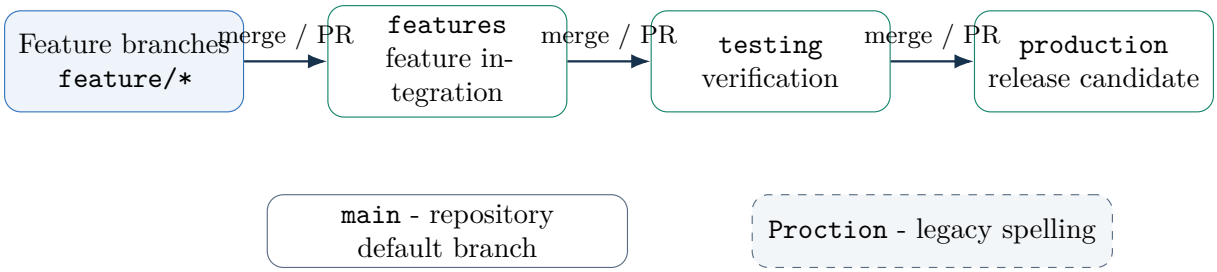


Figure 2.2: Intended promotion path for Jerry work. A feature branch is integrated, tested, then promoted to production.

Chapter 3

Installation and operation

3.1 Prerequisites

- Python 3.11 or later.
- PowerShell or another shell capable of running Python and Uvicorn.
- Project dependencies from `requirements.txt`.
- Optional service credentials only when enabling hosted LLM, hosted embeddings, Pinecone, or LangSmith.

3.2 Install dependencies and configure

Run commands from the repository root. A virtual environment is recommended.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
Copy-Item .env.example .env
```

On systems where the activation script is unavailable, activate the environment using the shell-specific command or invoke its Python executable directly. Keep the local `.env` file private and do not commit credentials.

3.3 Build the local sample index

In default local mode, no external API key is needed. Run the ingestion script before asking document questions:

```
python scripts/ingest_sample_data.py
```

The script loads Markdown metadata, creates overlapping chunks, embeds each chunk with the configured provider, and writes `data/local_index.jsonl`. The default chunk size is 900 characters with 120-character overlap. The bundled sample corpus currently has five fictional documents.

3.4 Start the API and interface

Run the API and Streamlit interface in separate terminals, both rooted at the repository directory.

```
uvicorn api.main:app --reload --host 127.0.0.1 --port 8010
```

```
streamlit run ui/streamlit_app.py --server.port 8501
```

Open http://127.0.0.1:8501/. The API root is http://127.0.0.1:8010/, health check is http://127.0.0.1:8010/health, and interactive OpenAPI documentation is http://127.0.0.1:8010/docs.

3.5 Run the development stack with Docker Compose

Docker Compose is an alternative local development workflow. Docker Desktop with Linux containers and the Compose plugin is required. The optional local `.env` file is read at runtime; it is excluded from the Docker build context and should not be committed. If no `.env` exists, the application defaults to local providers and demo tokens.

From the repository root in PowerShell, build the images and start the API and UI in the background:

```
if (-not (Test-Path .env)) { Copy-Item .env.example .env }
docker compose up --build -d
```

Create or refresh the sample index:

```
docker compose run --rm ingest
```

The ingestion service explicitly uses local embeddings and a local index; it does not require a Pinecone key. Open http://localhost:8501/; the API health endpoint and documentation are at http://localhost:8010/health and http://localhost:8010/docs. Stop and remove the development containers with:

```
docker compose down
```

The source bind mount supports development reloads. The defaults use host ports 8010 and 8501; when either host port is occupied, set `JERRY_API_PORT` or `JERRY_UI_PORT` before invoking Compose. Inside the Compose network, the UI must use `http://api:8010` as the backend URL.

3.6 Use the chat interface

1. Select a demo account in the sidebar. Its demo token is filled automatically and shown masked.
2. Ask about a document in the sample corpus, for example: “What are the policies?” or “What caused the settlement delay?”
3. Review the response. Open **Sources** to inspect the cited source documents.
4. Expand **Agent activity and observability** to inspect graph events.

5. Ask a short follow-up in the same session, or choose **Clear conversation memory** to start over.

Figure 3.1 shows the application header and question field. The screenshot is a genuine project capture, not a rendered mockup.

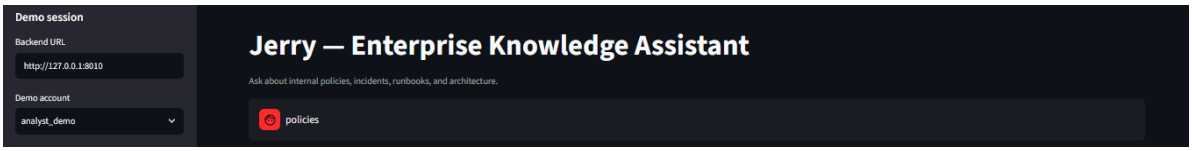


Figure 3.1: Jerry chat title and a submitted sample question.

The sidebar selects a demo account, shows the API URL, and keeps the demo token masked (Figure 3.2).

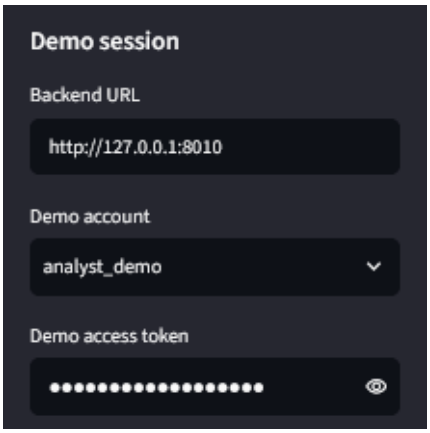


Figure 3.2: Demo session controls. The displayed identity is a demo account, not a production user.

3.7 Refresh or select an index backend

Local ingestion is the default. Pinecone can be selected with `-backend pinecone` for ingestion and `VECTOR_BACKEND=pinecone` for retrieval. Configure Pinecone API key, index, namespace, and a vector dimension matching the selected embedding provider. The remote index must exist and contain the ingested chunks before retrieval.

Chapter 4

Configuration

Settings are read from the process environment and, when present, a local `.env` file. The example template should retain local providers and tracing disabled until an operator explicitly configures an external service.

Variable	Local default	Purpose / notes
`APP_NAME`	`Jerry`	Application label.
`APP_ENV`	`development`	Environment label for logs.
`API_HOST`	`127.0.0.1`	API bind address.
`API_PORT`	`8010`	API port.
`API_BASE_URL`	`http://127.0.0.1:8010`	Backend URL used by the Streamlit client.
`LOG_LEVEL`	`INFO`	Python logging threshold.
`EMBEDDING_PROVIDER`	`local`	Local feature-hash or optional hosted semantic embedding adapter.
`OPENAI_API_KEY`	empty	Secret for optional OpenAI chat or embedding APIs. Do not commit.
`OPENAI_EMBEDDING_MODEL`	`text-embedding-3-small`	Embedding model used when the OpenAI adapter is selected.
`LLM_PROVIDER`	`local`	Local extractive response or optional OpenAI chat generation.
`OPENAI_CHAT_MODEL`	`gpt-4o-mini`	Model used by the optional OpenAI chat adapter.
`VECTOR_BACKEND`	`local`	Local JSONL index or Pinecone.
`PINECONE_API_KEY`	empty	Secret for optional Pinecone access.
`PINECONE_INDEX`	`jerry-demo`	Pinecone index name when selected.
`PINECONE_NAMESPACE`	`jerry-demo`	Pinecone namespace when selected.
`VIEWER_DEMO_TOKEN`	demo fallback	Viewer demo credential. Replace before any non-local use.

Variable	Local default	Purpose / notes
ANALYST_DEMO_TOKEN	demo fallback	Analyst demo credential. Replace before any non-local use.
ADMIN_DEMO_TOKEN	demo fallback	Administrator demo credential. Replace before any non-local use.
RATE_LIMIT_CAPACITY	20	Token bucket burst capacity per demo user.
RATE_LIMIT_REFILL_PER_MINUTE	10	Token refill rate per minute per demo user.
RETRIEVAL_TIMEOUT_SECONDS	15	Configured retrieval timeout.
LLM_TIMEOUT_SECONDS	30	Configured LLM timeout.
TOOL_TIMEOUT_SECONDS	3	Timeout for local MCP-style tool calls.
LANGSMITH_TRACING	false	Tracing is opt-in and also requires an API key.
LANGSMITH_API_KEY	empty	LangSmith secret. Never add a real key to the repository.
LANGSMITH_PROJECT	jerry-assistant	Project name for optional traces.
LANGSMITH_ENDPOINT	hosted LangSmith API	Optional regional or self-hosted endpoint override.

Chapter 5

Document ingestion and data model

5.1 Source records

`models/documents.py` defines validated `DocumentRecord` and `DocumentChunk` models. A document includes a stable document ID, title, text, department, document type, access level, and optional creation date. The current access-level values are `public`, `internal`, and `confidential`. Sample metadata is loaded from Markdown header fields.

5.2 Chunking and indexing

Text is whitespace-normalized, split into bounded chunks, and given a configurable overlap. Chunk IDs combine the source document ID and a chunk number. Each index record stores the chunk ID, embedding vector, source text, and retrieval metadata. The local index is a JSONL file intended to be inspectable. Pinecone ingestion uses the same chunk metadata and a configured namespace.

The sample corpus comprises the following fictional reference documents:

Document ID	Type / department	Subject
`POL-SEC-003`	Security policy / Security	Internal information handling and access responsibilities.
`INC-2025-014`	Incident / Payments	Delayed settlement after gateway timeouts and queue saturation.
`INC-2025-022`	Incident / Payments	Duplicate pending authorizations after retries without stable idempotency keys.
`ARC-PAY-002`	Architecture / Engineering	Payment API, queue, worker, gateway, and idempotency design.
`RUN-PAY-001`	Runbook / Payments	Operational checks for elevated payment latency and outages.

Chapter 6

Search and retrieval

6.1 Search modes

Sparse search uses BM25 over the local sample corpus and normalizes simple plurals such as "policies" to "policy". Dense search reads local vectors or queries Pinecone. When semantic embeddings are configured, the dense and sparse rankings are combined with weighted reciprocal rank fusion (RRF). Raw BM25 and dense similarity scores have different scales; RRF combines ranks rather than adding the raw scores.

With `EMBEDDING_PROVIDER=local`, feature-hash vectors are not treated as semantic rankings. The local path therefore uses BM25 alone to avoid arbitrary hash collisions. To use semantic dense retrieval, configure a semantic embedding provider, ingest again, and ensure the Pinecone index dimension matches if Pinecone is enabled.

6.2 Metadata filters and citations

Retrieval filters can constrain department, document type, access level, and inclusive creation-date bounds. Search results carry the source document ID, title, chunk ID, metadata, rank, and component scores. Citation validation only accepts source document IDs among the retrieved evidence.

Chapter 7

Assistant workflow and analysis

7.1 Grounded chat graph

The graph validates the request, resolves intent, routes to knowledge retrieval or a selected tool path, assesses whether evidence supports an answer, optionally performs one bounded wider retrieval retry, creates a response, and validates citations. It does not loop indefinitely. In local generation mode, the response is extractive and based on retrieved passages. Optional OpenAI generation is also constrained to retrieved evidence; prompt instructions label retrieved content as untrusted data.

If evidence remains insufficient, Jerry returns a safe no-evidence response rather than inventing policy or operational facts. This is expected when a question is outside the indexed corpus or the index is empty.

7.2 Recursive-analysis path

Analysis-shaped requests are routed through a bounded RLM-style path. It plans a focused incident search, restricts retrieval to incident documents, applies department and date filters when specified, splits passages into batches of at most three, and processes no more than four batches (12 passages). Python helper logic aggregates deterministic cause-pattern signals and source citations.

This is a controlled analysis demonstration, not an autonomous recursively calling language model. Cause labels are keyword and phrase pattern indicators. The returned answer and activity events should be interpreted with that limitation.

Chapter 8

Session memory, roles, and tools

8.1 Session memory

The in-process, thread-safe memory store retains at most the latest three user/assistant exchanges for a session and is bounded to 1,000 sessions. Short follow-ups can include the latest prior user question to resolve references. Prior assistant text is context, not factual evidence. Session identity is bound to the server-resolved user and role; a conflicting identity is rejected. Memory resets on API process restart and is not shared across multiple workers.

8.2 Role-based access control

The API validates the `X-Demo-Token` header, maps it to a server-side demo identity, and derives the role. The request body does not choose its role. Permission checks are placed at tool execution boundaries; the graph also blocks Viewer analysis before entering the RLM path.

Role	**Chat / search**	**Python analysis**	**MCP-style tools**	**Admin tools**
Viewer	Yes	No	No	No
Analyst	Yes	Yes	Yes	No
Administrator	Yes	Yes	Yes	Yes

Table 8.1: Demo role capability matrix.

The account selector is a convenience for the local demo, not a production login page or identity provider. Fallback tokens in the example configuration are public demo values; replace them and use an approved authentication system before exposing the service.

8.3 Tools

Knowledge search returns source-linked results. Python analysis accepts one to 12 validated passages and extracts bounded cause signals. Employee and service lookups return fictional local records through an MCP-client-style wrapper. They are not connected to a separate MCP server. The index-status endpoint is administrator-only.

Chapter 9

API reference

Except for root and health routes, API access uses `X-Demo-Token`. Chat bodies have a query of 1 to 4,000 characters and an optional session ID of 1 to 120 characters. The session ID defaults to `demo` if omitted.

Method	Path	Access	Purpose
GET	`/`	Public	Service status and links to health/docs.
GET	`/health`	Public	Health check.
GET	`/docs`	Public	Interactive OpenAPI documentation.
POST	`/api/chat`	Demo token	Non-streaming answer, citations, activity, memory count, and trace metadata.
POST	`/api/chat/stream`	Demo token	Server-sent events for node activity followed by the final response.
DELETE	`/api/chat/memory?session_id=...`	Demo token	Clear the current user's session memory.
POST	`/api/tools/search`	Demo token + search permission	Search with bounded top-k and optional department/type filters.
POST	`/api/tools/analysis`	Analyst or Administrator	Structured analysis over validated passages.
GET	`/api/tools/employees`	Analyst or Administrator	Local fictional employee lookup.
GET	`/api/tools/services`	Analyst or Administrator	Local fictional service lookup.
GET	`/api/tools/admin/index-status`	Administrator	Local index existence/status.

9.1 Chat request example

```
{
  "query": "What are the policies?",
  "session_id": "demo-session"
}
```

Supply the selected demo token in the HTTP header, not in the JSON body. A successful chat response includes `answer`, `citations`, `activity`, `memory_turns`, `trace_id`, and tracing status. The exact response fields are defined in `models/api.py`.

Chapter 10

Security and guardrails

10.1 Trust boundaries

- Request schemas bound query and tool-argument sizes. Graph validation rejects malformed control characters and recognized instruction override, secret extraction, permission bypass, and arbitrary command patterns.
- Retrieved passages are checked for source identifiers, size, and unsafe patterns before generation or analysis. Retrieved documents are treated as data, not executable instructions.
- Tool arguments are schema-validated, and role permissions are checked immediately before execution.
- Responses are length-bounded, token-like secrets are redacted, and citations are checked against retrieved document IDs. Invalid citations are removed; a cited extractive fallback is available if a grounded response omitted citations.

10.2 Limitations and safe use

The injection detector uses phrase patterns. Obfuscation and novel attack wording may evade it; benign passages may also match. These controls are defense in depth and are not a security guarantee. Demo users, local tokens, process memory, and local logs are not a production security architecture. Do not use real customer or confidential information in this assignment demo. LangSmith can retain model inputs, outputs, and retrieved evidence when tracing is enabled; send synthetic data unless appropriate organizational controls approve otherwise.

Chapter 11

Reliability and observability

11.1 Rate limits

Authenticated chat and tool routes use a per-user token bucket. Defaults allow a burst of 20 requests and refill 10 tokens per minute. A rejected request returns HTTP 429 and a `Retry-After` header. Root and health endpoints remain available. The limiter is process-local; multi-worker or multi-host deployments need a shared state store such as Redis.

11.2 Timeouts and error mapping

Retrieval, LLM, and tool timeouts are configurable. A tool timeout returns a safe timeout response. OpenAI failures can fall back to cited extractive output. Unexpected errors are logged with a trace ID while the client receives a generic message.

HTTP status	**Typical meaning**
401	Missing or invalid demo token.
403	Authenticated demo role lacks permission for the requested tool.
409	A memory session is being reused with a conflicting user or role identity.
422	Request or tool parameters do not satisfy the schema or size constraints.
429	Per-user request limit reached; inspect `Retry-After`.
500	Unexpected server failure; provide the trace ID to the maintainer.
503	Assistant or downstream service is unavailable.
504	A bounded MCP-style tool call exceeded its timeout.

Table 11.1: Common error responses. Exact details can depend on the route and failure.

11.3 LangSmith and streamed activity

Tracing is enabled only when `LANGSMITH_TRACING=true` and a non-empty `LANGSMITH_API_KEY` are configured. Graph runs carry a name, tags, role/user/session metadata, and trace ID. Retrieval, Python analysis, and local MCP-style wrappers can appear as child runs. The Streamlit client consumes `POST /api/chat/stream`, which emits activity events as server-sent events and then a final response. Non-streaming clients can use `POST /api/chat`.

The event panel can be reviewed in Figure 11.1. It represents an earlier captured run; the displayed no-evidence outcome is not the expected answer for every query.

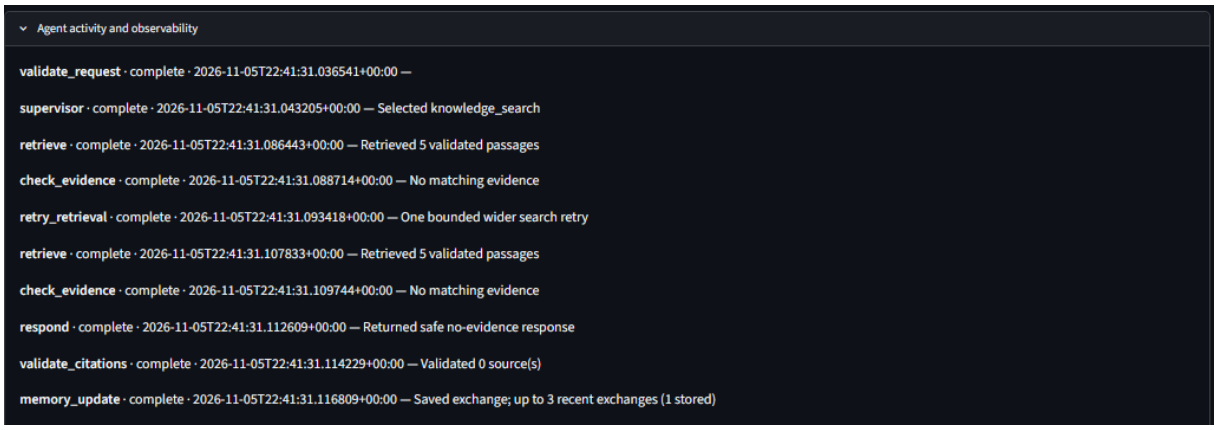


Figure 11.1: Expanded activity panel showing graph steps.

Chapter 12

Testing and verification

12.1 Recorded local result

The test record dated November 6, 2026 reports 10 feature areas plus the Streamlit UI smoke check: 11 checks passed and zero failed. The local API root, health endpoint, OpenAPI docs, and Streamlit page each returned HTTP 200. The local sample corpus contained five documents and five indexed chunks.

No.	Feature	Recorded check	Result
1	Project foundation	Root, health, and API docs respond; protected tools reject a missing token with 401.	PASS
2	Ingestion and data model	Five Markdown documents loaded and five metadata-bearing chunks generated.	PASS
3	Hybrid retrieval	Policy query returns `POL-SEC-003`; department filtering returns payment documents.	PASS
4	LangGraph flow	Grounded chat returns a citation and workflow activity events.	PASS
5	Recursive analysis	Analyst incident summary returns RLM activity/citations; batches are at most 3 and capped at 4.	PASS
6	Session memory	Follow-up retains session context and clear-memory succeeds.	PASS
7	RBAC and tools	Viewer denied analysis/admin; Analyst analysis and employee tool allowed; Administrator index status allowed.	PASS
8	Security	Injection rejection, retrieved-content screening, secret redaction, and invalid-citation removal exercised.	PASS
9	Resilience	Oversized query returns 422; rate-limited request returns 429 with `Retry-After`.	PASS
10	Observability	Streaming activity and final events plus trace ID verified. Remote export was disabled.	PASS*
UI	Streamlit interface	AppTest renders title and chat input without app exception.	PASS

** Local stream and trace-ID behavior was exercised; no LangSmith trace was sent.*

12.2 Coverage not established

The recorded run did not contact Pinecone, hosted semantic embeddings, an external MCP server, or the LangSmith service. It does not establish retrieval performance on a larger corpus, multi-worker behavior, deployment security, or production availability. AppTest emitted its standard bare-mode `ScriptRunContext` warning but no application exception.

12.3 Docker development verification

On November 7, 2026, the Docker Compose development path was checked on Windows with Docker Desktop 29.9.2 and Compose 5.6.1. The images built successfully, both application services ran, and the API health check reported healthy.

Check	Observed result
Docker Engine / Compose versions	29.9.2 / 5.6.1
Compose image build	PASS; API and UI images built
API container health	Healthy
`GET /health`	HTTP 200
`GET /docs`	HTTP 200
Streamlit at host port 8501	HTTP 200
UI container to `http://api:8010/health`	HTTP 200; Compose DNS path works
Local ingestion service	PASS; 5 sample documents and 5 chunks indexed
Authenticated chat request	HTTP 200; 1 citation and a trace ID returned

These results establish a local development smoke test only. The trace ID does not by itself establish remote LangSmith delivery, and this run does not assess production security or external provider integrations. The detailed record is `docs/testing/docker-testing.md`.

12.4 Repeat the manual checks

1. Run `python scripts/ingest_sample_data.py` from the repository root.
2. Start Uvicorn on port 8010 and Streamlit on port 8501 in separate terminals.
3. Open http://127.0.0.1:8501/, select a demo role, and ask “What are the policies?”
4. Confirm the source citation, then use Analyst for analysis and local tool checks and Administrator for index status.
5. Use http://127.0.0.1:8010/docs for request schemas and route exploration.

The detailed record and known limitations are maintained in `docs/testing/testing.md` and `docs/testing/role-based-testing.md`.

Chapter 13

Troubleshooting

Symptom	Action
Streamlit cannot connect	Keep both processes running. Check http://127.0.0.1:8010/health and confirm the sidebar Backend URL is correct.
No supporting documents	Run the ingestion script from the repository root. Verify the local index exists and try a question matching one of the five sample documents.
`ModuleNotFoundError` for a project package	Launch Streamlit from the repository root with `streamlit run ui/streamlit_app.py`.
HTTP 401	Verify the selected demo token is present and matches the configured server-side token.
HTTP 403	Use a role permitted to perform the operation. This is expected for Viewer requests to analysis or admin tools.
HTTP 409	Start a fresh session if the user identity or role has changed.
HTTP 422	Check required fields, text lengths, and the schema in http://127.0.0.1:8010/docs.
HTTP 429	Wait for the per-user token bucket to refill; inspect `Retry-After`.
Docker port already in use	Change `JERRY_API_PORT` or `JERRY_UI_PORT`, then restart the Compose stack.
UI cannot reach API in Docker	Set the UI backend URL to `http://api:8010`; `localhost` inside a container is that same container.
Tracing disabled	Configure the explicit enable flag and key in a private `.env` file. Do not commit the key.

Chapter 14

Deployment considerations

Before using any hosted environment, replace fallback demo tokens, introduce a production identity provider, protect secrets with a managed secret store, and configure TLS and network access controls. Move session state and rate-limit buckets to shared durable infrastructure when using multiple workers. Establish document-level authorization and audit policy for real information. Validate prompt-injection controls with the organization's threat model. Decide what telemetry can be sent to LangSmith and configure retention/region settings accordingly. Configure and test each external provider independently. The current implementation and test evidence are for a local assignment/demo, not a production readiness certification.

Chapter 15

Glossary and source documents

15.1 Glossary

BM25	A keyword-ranking function used by the sparse local search implementation.
Chunk	A bounded text section derived from a source document and retained with source metadata.
LangGraph	The graph orchestration library used to construct and stream assistant workflow nodes.
LangSmith	Optional tracing and observability service for graph and tool executions.
MCP	Model Context Protocol. Jerry's current employee/service integrations are local MCP-style wrappers, not a remote MCP server.
RLM-style	A bounded recursive-analysis-inspired process in this project; it uses focused retrieval, capped passage batches, and deterministic Python analysis.
RBAC	Role-based access control: tool permissions are derived from the server-resolved role.
RRF	Reciprocal rank fusion, a method for combining ranked candidate lists.
SSE	Server-sent events, used by the streaming chat endpoint to report node updates and the final response.

15.2 Repository documentation references

This manual consolidates information from the following repository sources. They remain the detailed, component-level notes:

- `docs/architecture.md` and `docs/development-flow.md`
- `docs/ingestion.md` and `docs/retrieval.md`
- `docs/assistant-flow.md` and `docs/recursive-analysis.md`
- `docs/session-memory.md` and `docs/testing/role-based-testing.md`
- `docs/security-guardrails.md` and `docs/resilience.md`
- `docs/observability.md` and `docs/testing/testing.md`

- `Dockerfile`, `docker-compose.yml`, and `docs/testing/docker-testing.md`

Appendix A

Compile this LaTeX source

Install a LaTeX distribution that includes pdfLaTeX and the packages listed in the preamble (including TikZ). From the repository root, compile twice so the table of contents and cross-references settle:

```
pdflatex -interaction=nonstopmode docs/jerry-software-documentation.tex
pdflatex -interaction=nonstopmode docs/jerry-software-documentation.tex
```

The resulting PDF is written next to the source. Figure paths are relative to the repository root. The PDF contains static screenshots; the animated walkthrough remains in the README as a GIF.